Few-Shot Residual Adaptation of Pretrained Latent World Models

Jakub Suliga
Institute for Anthropomatics and Robotics
Intuitive Robots Lab
Karlsruhe Institute of Technology
jakub.suliga@student.kit.edu

Pankhuri Vanjani
Institute for Anthropomatics and Robotics
Intuitive Robots Lab
Karlsruhe Institute of Technology
pankhuri.vanjani@kit.edu

Nisarga Nilavadi
Department Computer Science & Artificial Intelligence
University of Technology Nuremberg
nisarga.nilavadi@utn.de

Wolfram Burgard
Department Computer Science & Artificial Intelligence
University of Technology Nuremberg
wolfram.burgard@utn.de

Rudolf Lioutikov
Institute for Anthropomatics and Robotics
Intuitive Robots Lab
Karlsruhe Institute of Technology
rudolf.lioutikov@kit.edu

Abstract

Pretrained latent world models can support visual planning and control, but their predictions may fail when physical dynamics change at deployment. We study whether such models can adapt from only a few interactions without modifying their pretrained weights or identifying the changed physical parameters. We introduce a few-shot residual adaptation approach that learns self-supervised corrections from latent prediction errors. The corrections are applied both inside the latent predictor and directly to its predicted latent state, while the pretrained world model remains frozen. We evaluate the approach in simulation and on a real robot under center-of-mass and friction shifts. These changes reduce the success of the frozen model to near zero. Using three interaction episodes for adaptation, our dual-site approach recovers 90% success under the center-of-mass shift and 100% under the modified friction on the real robot. Ablations show that correcting both locations outperforms single-site adaptation. Our diagnostics further suggest that much of the learned correction reduces a general prediction mismatch, while successful control also depends on shift-specific information captured during adaptation.

1 Introduction

World models predict how observations evolve under robot actions and have been used to support planning and control [7, 4, 25]. Latent world models are particularly promising because they can

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: 8th Robot Learning Workshop: Is Physical AI Going Zero-Shot?.

learn task-relevant representations without predicting every pixel, enabling efficient model-based control from visual observations [1, 29, 5]. A central challenge is that a model trained under one set of physical conditions may be deployed under another. Changes in center of mass, friction, payload, actuator properties, or contact dynamics can alter the effect of the same action even when the task, visual appearance, and goal remain unchanged [14, 15, 2]. Existing approaches often address this mismatch by retraining the model [13], estimating physical parameters [14, 2], adapting a policy [15, 11], or randomizing dynamics during training [18]. These strategies can be effective, but they may require substantial data, access to the training procedure, a suitable parameterization of the dynamics, or prior knowledge about the deployment shift.

In this work, we study how a pretrained latent world model can be efficiently adapted from a few interactions. We keep the pretrained model frozen and learn lightweight residual corrections from its prediction errors. These corrections are trained self-supervised from observed transitions, requiring neither rewards nor knowledge of the changed physical parameters. We inject them at two points in the latent dynamics pipeline: within the predictor and on its predicted latent state. We evaluate this setting in simulation and on a real robot under shifted center-of-mass and modified friction setting. These changes reduce the success of the frozen world model to near zero, while adaptation from only three interaction episodes recovers most of the lost performance. We find that adapting both inside the predictor and at its output works better than adapting either location alone. Our diagnostics further show that much of the correction reduces a general prediction mismatch, while the large control gap between general- and shifted-dynamics adapters suggests that some of the learned correction depends on the dynamics seen during adaptation.

Our main contributions are:

1. **Dynamics-shift failure:** We show that physical dynamics shifts can break control while the task and controller remain fixed.
2. **Few-shot residual adaptation:** We restore control from a few self-supervised interactions while keeping the pretrained world model frozen.
3. **Dual-site correction:** We show that adapting both inside the predictor and on its output outperforms single-site alternatives.

2 Related Work

JEPA-style latent world models Several works investigate action-conditioned prediction for visual planning. V-JEPA 2-AC, a joint-embedding predictive architecture, predicts future latent representations conditioned on robot actions [1]. Similarly, DINO-WM predicts future DINOv2 patch representations from observation and action histories and employs model-predictive control to reach visual goals [29]. Other latent world models, including DreamerV3 [4], TD-MPC2 [5], and DayDreamer [25], learn compact predictive representations for control, but generally couple world-model learning with task-specific reward or value prediction.

Parameter-efficient and test-time adaptation Parameter-efficient fine-tuning (PEFT) adapts a pretrained network while keeping most parameters frozen. LoRA uses low-rank weight updates [9], bottleneck adapters add nonlinear residual branches [8], and Adapter$^+$ scales such a branch after the feed-forward block [21]. Other work instead corrects the network output [28]. Test-time adaptation updates deployed models from unlabeled data using self-supervised or entropy-based objectives [22, 23, 6]. We combine these ideas by learning lightweight residual corrections from deployment interactions while keeping the pretrained world model frozen, and study whether dynamics shifts are better corrected inside the latent predictor, at its output, or both.

World model adaptation to dynamics shifts Prior work follows three main strategies. *Explicit identification* estimates physical parameters: PIN-WM [14] fits inertia, friction, and restitution through a differentiable simulator, while Incremental Few-Shot Adaptation [2] identifies simulator parameters from interaction data. Both require a simulator and suitable parameterization. *Implicit conditioning* infers dynamics from interaction history. DyWA [15] conditions its predictor on a learned dynamics embedding, but remains limited to shifts represented during training. *Direct adaptation* uses prediction errors at deployment. AdaJEPA [24], Simulation Distillation [13], and AdaWorldPolicy [27] update pretrained weights, whereas ReDRAW [12] and SPREAD [16] keep the

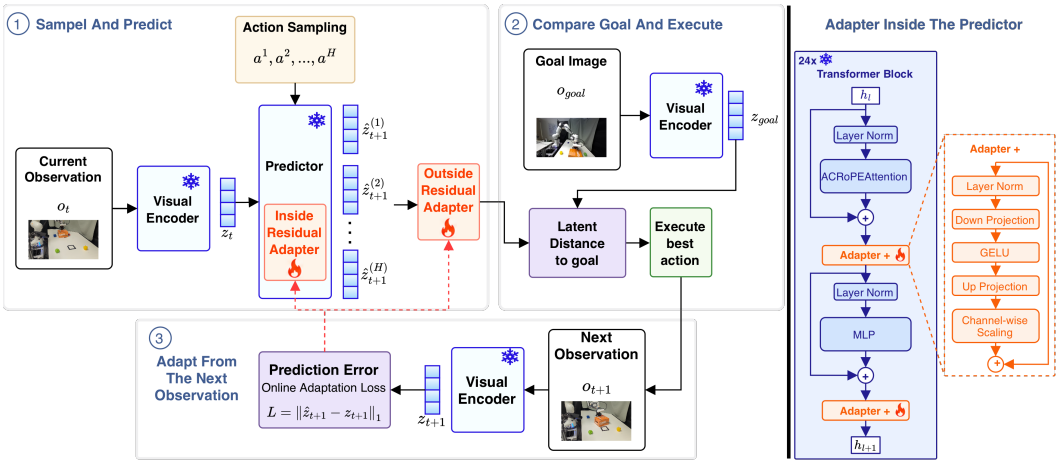


Figure 1: Dual-site latent correction in a frozen world model. The frozen predictor rolls out sampled actions, corrected inside every block and once more on its output. The CEM controller ranks the corrected predictions by latent distance to the encoded goal and executes the best action. The next observation supplies the target of the self-supervised loss that trains both residuals.

backbone frozen but adapt only inside the predictor. In contrast, we keep the pretrained model frozen and learn self-supervised residual corrections both inside the predictor and on its output.

3 Method

We adapt a pretrained latent world model to the dynamics of a deployment scene without changing any of its original parameters (Figure 1). The pipeline alternates model-predictive control with self-supervised adaptation over three interaction episodes. Our dual-site adapter corrects the model in two places, inside its latent predictor and at the predictor output. Both corrections are frozen once adaptation ends, and the original planning procedure is retained throughout.

3.1 Latent world model and planning

Our base model is DUET-DINO [17], a variant of V-JEPA 2-AC [1]. It predicts action-conditioned latent transitions from a static side camera and a wrist camera. A frozen DINOv3 encoder maps each 224×224 image to 14×14 patch tokens of dimension 1280, and cross-attention exchanges information bidirectionally between the two views [20]. Separate side- and wrist-view heads predict the next latent tokens of each view. Each head comprises 24 frame-causal Transformer blocks of width 1024, conditioned on the seven-dimensional end-effector state $\mathbf{s}_t$ and action $\mathbf{a}_t$. The gripper is held closed throughout, so that last dimension is constant and is neither planned nor predicted. Writing $\mathbf{z}_t^v$ for the encoded observation of view $v \in \{\mathrm{s}, \mathrm{w}\}$, the one-step prediction is

$$\widehat{\mathbf{z}}_{t+1}^v = \mathrm{LN}(f_\theta^v(\mathbf{z}_t^{\mathrm{s}}, \mathbf{z}_t^{\mathrm{w}}, \mathbf{s}_t, \mathbf{a}_t)), \tag{1}$$

with θ the pretrained cross-attention and predictor parameters.

Planning uses a one-step cross-entropy method (CEM) controller [19]. At each robot step, it samples actions, predicts the corresponding side and wrist-view latents, and ranks them by latent distance to the encoded goal. After two CEM iterations, the proposal mean is executed. We keep this controller fixed and adapt only the latent predictions it consumes.

3.2 Dual-site latent correction

A dynamics shift changes the action-conditioned transition that the pretrained predictor must model. We therefore correct both its internal computation and its final latent prediction, while keeping the encoder, cross-attention blocks, and all pretrained predictor weights frozen.

Inside the predictor. We insert Adapter$^+$ modules at two points in every block of both predictor heads, after the attention residual and after the completed block [21]. For a hidden token $\mathbf{h} \in \mathbb{R}^{1024}$,

each module computes

$$\mathbf{h}' = \mathbf{h} + A(\mathbf{h}), \qquad A(\mathbf{h}) = \mathbf{c} \odot \mathbf{W}_{\text{up}}\,\text{GELU}(\mathbf{W}_{\text{down}}\text{LN}(\mathbf{h}))\,, \tag{2}$$

with $\mathbf{W}_{\text{down}} \in \mathbb{R}^{128\times1024}$, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{1024\times128}$ and a learned channel scale $\mathbf{c} \in \mathbb{R}^{1024}$ that sets the strength of the residual per feature. Two sites across 24 blocks and two heads give 96 modules in total. They can reshape the intermediate computation without touching the pretrained backbone.

Outside the predictor. A shared token-wise residual MLP then corrects whatever the adapted predictor produces, $\widetilde{\mathbf{z}}^v_{t+1} = \widehat{\mathbf{z}}^v_{t+1} + R(\widehat{\mathbf{z}}^v_{t+1})$ for either view. It is $1280 \to 1024 \to 1024 \to 1280$ with GELU activations and no input normalization, and sees only the predicted token, not the previous latent or the action. Sharing it across views supplies one common output-space correction, while the view-specific internal adapters keep separate capacity.

We zero-initialize the up-projection of every internal adapter and the final layer of the external MLP, so both residuals vanish before the first update and the adapted model starts out equivalent to the frozen base model. Together the internal adapters (25.6M parameters) and the external MLP (3.7M) add 29.2M trainable parameters, 4.53% of the frozen predictor.

3.3 Self-supervised online adaptation

The adapters learn only from prediction errors between predicted and observed transitions. Each episode is subsampled to 4 Hz, encoded once with the frozen encoder, and the resulting layer-normalized latents are cached in a replay buffer. The next encoded observation serves as the target for the one-step objective:

$$\mathcal{L}_{\text{pred}} = \sum_{v\in\{\text{s,w}\}} \left\|\widetilde{\mathbf{z}}^v_{t+1} - \mathbf{z}^v_{t+1}\right\|_1 . \tag{3}$$

Neither rewards nor knowledge of the changed physical parameters are required.

After each of the first three episodes, we update the adapters sequentially in two phases. First, the internal Adapter$^+$ modules are trained for 25 steps on batches of eight 8-frame clips using teacher-forced and two-step autoregressive losses over both views. Clips come from the current episode initially and the replay buffer thereafter. We then freeze these adapters, recompute one-step predictions, and train the external MLP for 25 steps on batches of up to 256 transitions with an ℓ_1 loss. Both phases use AdamW with learning rates 4.343×10^{-4} (inside) and 1.234×10^{-3} (outside).

4 Evaluation

We change only the physical dynamics while keeping the task, goal, and planning fixed. Our evaluation addresses four research questions: **(RQ1)** How do physical dynamics shifts affect planning with a pretrained latent world model?, **(RQ2)** How much of the planning can a few-shot adaptation recover?, **(RQ3)** How does correction placement affect adaptation?, and **(RQ4)** What does the learned adaptation capture?

4.1 Setup

Task and dynamics shifts. We study goal-directed box pushing, where the robot uses its closed gripper to push a box to a target pose. Under *general* dynamics, the box has a centered center of mass and uniform friction between the box and the tabletop, so a forward push moves it toward the goal without rotation. We evaluate on two shifts while keeping the task fixed. In the *center-of-mass* (CoM) shift, a hidden weight moves the center of mass to the right. In the modified *friction*, a cloth under half of the track creates asymmetric friction across the box. Note that the scene stays visually unchanged under the CoM shift, whereas the cloth of the modified friction is visible.

Platforms. *Real robot:* We use a Franka Panda in the DROID configuration with a static side camera and a wrist camera [10]. Model inferences are run on a single RTX 4060 Ti. Each control step takes about 28 sec end-to-end, and the three adaptation episodes contain only 24 frames in total. We evaluate both dynamics shifts. *Simulation:* We reproduce the same two-view tabletop setup in the RoboLab simulator [26]. Each episode lasts 25 steps, yielding 75 frames across three adaptation episodes. We evaluate only the center-of-mass shift because the split-friction surface could not be reproduced faithfully in RoboLab.

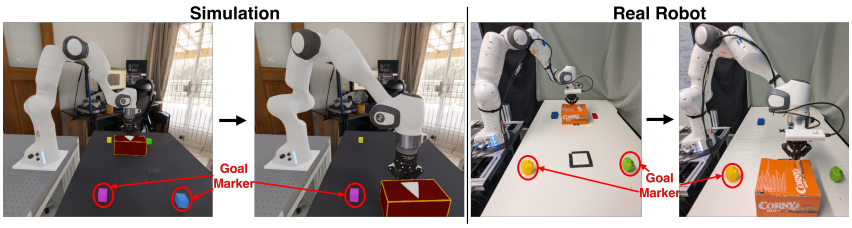


Figure 2: The pushing task in simulation and on the real robot, each shown at the start of an episode and after a successful push.

Protocol. We use the same CEM planning in simulation and on the real robot, except for the maximum action norm: 0.25 in simulation and 0.2 on the real robot [19]. In simulation, we additionally evaluate a Diffusion Policy [3] as a non-world-model baseline. An episode is successful if at least half of the box crosses the goal marker, which corresponds to 8 cm on the real robot and 4.7 cm in simulation, while its orientation stays within about $25°$ of the starting orientation (see Figure 2). For the architecture comparison in simulation, we report 150 runs per method across three adaptation seeds, with 50 runs per seed. Each real-robot experiment uses 30 runs.

4.2 Results

RQ1: How do physical dynamics shifts affect planning with a pretrained latent world model? As shown in Table 1, under general dynamics, Diffusion Policy achieves 96.0% success in simulation [3] and the frozen DUET-DINO model reaches 85.3% in simulation and 86.7% on the real robot. After shifting the box's center of mass, success drops to 0% for all baselines. After changing friction, the frozen DUET-DINO model reaches only 3.3% on the real robot. This shows that changes in object properties lead to changes in physical dynamics, which in turn break planning.

Table 1: Aggregated success rate over 150 runs under general dynamics and the two dynamics shifts. Diffusion Policy is evaluated in simulation only.

	Success rate (%) ↑		
Method	**General**	**Shifted CoM**	**Modified friction**
Simulation			
DUET-DINO	85.3	0.0	–
Diffusion Policy	96.0	0.0	–
Real robot			
DUET-DINO	86.7	0.0	3.3

RQ2: How much of the planning can a few-shot adaptation recover? The frozen DUET-DINO model achieves 0.0% success under the center-of-mass shift and 3.3% under the modified friction (Table 1). Shift-specific full finetuning and LoRA recover some performance (Table 2), but vary substantially across the two shifts. In contrast, our shift-specific dual-site adapter reaches 90.0% success under the center-of-mass shift and 100.0% under the modified friction. Few-shot adaptation therefore recovers most of the lost planning performance across both evaluation setups.

RQ3: How does correction placement affect adaptation? We compare corrections inside the predictor, on the predicted latent state, and at both sites. As shown in Table 2, no single-site method fully recovers planning: the best internal method, LoRA, reaches 32.7% success, while the best external correction reaches 24.0%. Our dual-site adapter performs substantially better at 56.7%, with sequential training also outperforming joint training (56.7% vs. 44.0%). This suggests that correcting both the predictor computation and its output is important. The two sites also show complementary failure modes: internal adaptation often moves in the correct direction but can be unstable, while the external residual gives more consistent motion but may move in the wrong direction. This helps explain the stronger performance of dual-site adaptation.

RQ4: What does the learned adaptation capture? We analyze the learned correction on 306 held-out episodes (7,650 transitions). The general-dynamics adapter is trained on the general task with the

Table 2: Success rates in simulation and on the real robot. Simulation results are mean ± standard deviation over three seeds (50 runs each). The four best-performing adapters are evaluated on the real robot. Modified friction is evaluated only there as it was unreliable in RoboLab.

Method	Success rate (%) ↑		
	Simulation	Real robot	
	Shifted CoM	Shifted CoM	Modified friction
Frozen Base Policy			
Diffusion Policy	0.0 ± 0.0	–	–
DUET-DINO (frozen)	0.0 ± 0.0	0.0	3.3
W/ Adapters (Inside)			
Full finetuning	28.0 ± 10.0	20.0	**100.0**
LoRA	32.7 ± 9.0	30.0	66.7
Adapter$^+$	18.0 ± 5.3	–	–
Residual MLP	10.7 ± 9.0	–	–
Bottleneck adapter	12.7 ± 12.2	–	–
W/ Adapters (Outside)			
Residual MLP	24.0 ± 3.5	–	–
Channel-wise scaling	23.3 ± 7.6	–	–
W/ Adapters (Combination)			
Dual-site adapter, joint	44.0 ± 6.0	80.0	90.0
Dual-site adapter (Ours)	**56.7 ± 13.3**	**90.0**	**100.0**

Table 3: Panel (a) ablates dynamics transfer, adapter location, and domain transfer. The arrow separates the training condition from the test condition. Panel (b) varies the scene appearance at test time. The adapter is trained on the red box in the unchanged scene and never on these variations. All entries are success rates in percent over 50 runs from a single adaptation seed.

(a) Adapter ablations across dynamics specificity, correction site, and cross-domain transfer.

Variant / protocol	Success rate (%) ↑
Dynamics transfer (train → test)	
General dynamics → CoM shift	6.0
CoM shift → CoM shift	72.0
CoM shift → nominal task	54.0
Adapter location	
Wrist predictor only	18.0
Side-view predictor only	14.0
Domain transfer (train → test)	
Real robot → sim (50 steps)	10.0
Real robot → sim (25 steps)	0.0
Sim → real robot	0.0

(b) Appearance generalization

Variation	Success rate (%) ↑
Red box (seen)	
Original scene	72.0
New table	84.0
New marker	54.0
New background	60.0
Turquoise box (unseen)	
Original scene	74.0
New table	62.0
New marker	74.0
New background	60.0
New table + marker	54.0

same three-episode self-supervised procedure as our adapter, and it reduces prediction error almost as much as the CoM-shift adapter: it shares 95.3% of the side-view and 71.7% of the wrist-view error reduction. However, under the CoM shift it achieves only 6.0% success, compared with 72.0% for the CoM-shift adapter (Table 3). Thus, reducing prediction error alone is not sufficient for successful control. The large control gap suggests that the CoM-shift adapter captures additional shift-specific information that is important for control, although our analysis does not reveal the mechanism or physical quantity encoded by this correction. Adaptation also comes at a cost: the CoM-shift adapter reaches only 54.0% on the general task, compared to 85.3% for the frozen model. The adapter remains effective under changes in object color, table, marker, and background, but transfers poorly between simulation and the real robot (Table 3), suggesting that it learns a deployment-specific correction rather than a transferable physics model.

5 Conclusion and Limitations

We study few-shot adaptation of pretrained latent world models under changes in physical dynamics. We show how changes in center of mass or friction can break control even when the task and controller remain fixed. By keeping the pretrained model frozen and learning residual corrections from only a few observed transitions, our dual-site adaptation recovers most of the lost performance. Correcting both the predictor computation and its output consistently outperforms single-site adaptation. Our analysis further shows that reducing prediction error alone is not sufficient for successful control. The large gap between general- and shifted-dynamics adapters suggests that successful adaptation captures additional shift-specific information, although our analysis does not reveal the underlying mechanism or physical quantity represented by this correction. The adapter remains effective under visual changes but transfers poorly between simulation and the real robot, suggesting that it learns a deployment-specific correction rather than a generally transferable physics model.

Our current study is limited to center-of-mass and friction shifts in a pushing task and learns a separate residual correction for each dynamics setting. The adapted model may also lose performance when returned to general dynamics. Future work will extend this study to broader dynamics shifts, develop mechanisms for retaining and reusing multiple learned corrections, and investigate the mechanistic basis of these corrections and what dynamics-related information they encode.

References

[1] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[2] Fabian Baumeister, Lukas Mack, and Joerg Stueckler. Incremental few-shot adaptation for non-prehensile object manipulation using parallelizable physics simulators. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

[3] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.

[4] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[5] Nick Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.

[6] Nicklas Hansen, Rishabh Jangir, Yu Sun, Guillem Alenyà, Pieter Abbeel, Alexei A Efros, Lerrel Pinto, and Xiaolong Wang. Self-supervised policy adaptation during deployment. *arXiv preprint arXiv:2007.04309*, 2020.

[7] Bohan Hou, Gen Li, Jindou Jia, Tuo An, Xinying Guo, Sicong Leng, Haoran Geng, Yanjie Ze, Tatsuya Harada, Philip Torr, et al. World model for robot learning: A comprehensive survey. *arXiv preprint arXiv:2605.00080*, 2026.

[8] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*. PMLR, 2019.

[9] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[10] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

[11] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.

[12] JB Lanier, Kyungmin Kim, Armin Karamzade, Yifei Liu, Ankita Sinha, Kat He, Davide Corsi, and Roy Fox. Adapting world models with latent-state dynamics residuals. *arXiv preprint arXiv:2504.02252*, 2025.

[13] Jacob Levy, Tyler Westenbroek, Kevin Huang, Fernando Palafox, Patrick Yin, Shayegan Omidshafiei, Dong-Ki Kim, Abhishek Gupta, and David Fridovich-Keil. Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation. *arXiv preprint arXiv:2603.15759*, 2026.

[14] Wenxuan Li, Hang Zhao, Zhiyuan Yu, Yu Du, Qin Zou, Ruizhen Hu, and Kai Xu. Pin-wm: Learning physics-informed world models for non-prehensile manipulation. *arXiv preprint arXiv:2504.16693*, 2025.

[15] Jiangran Lyu, Ziming Li, Xuesong Shi, Chaoyi Xu, Yizhou Wang, and He Wang. Dywa: Dynamics-adaptive world action model for generalizable non-prehensile manipulation. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2025.

[16] Jihun Moon and Seong-Woo Kim. Spread: Scalable pre-trained world model for adaptive dynamics model. *IEEE Robotics and Automation Letters*, 2026.

[17] Nisarga Nilavadi, Ralf Römer, Moritz Reuss, Michael Krawez, Tobias Jülg, Angela P Schoellig, Rudolf Lioutikov, and Wolfram Burgard. Duet-dino: Simultaneous cross-view world modeling for latent planning in robot manipulation. *arXiv preprint arXiv:2609.10506*, 2026.

[18] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018.

[19] Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and computing in applied probability*, 1999.

[20] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.

[21] Jan-Martin O Steitz and Stefan Roth. Adapters strike back. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024.

[22] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*. PMLR, 2020.

[23] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*, 2020.

[24] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. Adajepa: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026.

[25] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*. PMLR, 2023.

[26] Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. Robolab: A high-fidelity simulation benchmark for analysis of task generalist policies. *arXiv preprint arXiv:2604.09860*, 2026.

[27] Ge Yuan, Qiyuan Qiao, Jing Zhang, and Dong Xu. Adaworldpolicy: World-model-driven diffusion policy with online adaptive learning for robotic manipulation. *arXiv preprint arXiv:2602.20057*, 2026.

[28] Andy Zeng, Shuran Song, Johnny Lee, Alberto Rodriguez, and Thomas Funkhouser. Tossingbot: Learning to throw arbitrary objects with residual physics. *IEEE Transactions on Robotics*, 2020.

[29] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.